

# LLMProvider

## LLMProvider

**Единый интерфейс, 43 провайдера — с встроенными механизмами размыкания цепи, повторными попытками и мониторингом работоспособности.**

### Краткое описание

LLMProvider — это универсальный, многократно используемый модуль Go, который определяет единый интерфейс LLMProvider и встроенные шаблоны отказоустойчивости: размыкатель цепи, мониторинг работоспособности, повторные попытки с экспоненциальным откатом, ленивую загрузку. Модуль предоставляет 43 конкретные реализации провайдеров за одним контрактом, а также универсальный адаптер, совместимый с OpenAI, и честный механизм обнаружения моделей без жёстко закодированных запасных вариантов.

### Краткое описание (сокращённо)

Многократно используемый модуль Go, предоставляющий единый интерфейс LLMProvider (Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig) с примитивами отказоустойчивости: размыкателем цепи, монитором работоспособности, повторными попытками с рандомизированным откатом, ленивой инициализацией. Включает 43 адаптера провайдеров и универсальный адаптер, совместимый с OpenAI. Потокбезопасен.

### Подробное описание

LLMProvider — это тот самый уровень абстракции, который необходим каждому сервису, использующему LLM, но который почти никто не реализует должным образом. Это невидимая, но критически важная инфраструктура,

отделяющая демонстрационный проект от системы, способной выдержать реальную нагрузку. Модуль определяет единый интерфейс с поддержкой всех необходимых возможностей — Complete, CompleteStream, HealthCheck, GetCapabilities, ValidateConfig — благодаря чему код приложения работает с одним контрактом независимо от того, какой из 43 бэкендов обрабатывает запрос. При этом модуль предоставляет инструменты эксплуатационной устойчивости, превращающие хрупкие вызовы провайдеров в нечто, что можно запускать в продакшене без постоянного напряжения.

Трёхстадийный размыкатель цепи (закрыто → открыто → полуоткрыто) прозрачно оборачивает любой провайдер — *включая его потоковый канал*, где пустой поток корректно расценивается как сбой. Это позволяет изолировать проблемный бэкенд, предотвращая падение всей системы. Центральный CircuitBreakerManager отслеживает состояние всех размыкателей одновременно. Настраиваемый монитор работоспособности непрерывно проверяет провайдеры, переводя их в состояния *работоспособен / деградировал / неработоспособен / неизвестно* на основе пороговых значений и интервалов проверок. Это позволяет замечать деградацию до того, как она приведёт к отказу.

Логика повторных попыток использует экспоненциальный откат с рандомизацией, принимая осознанные решения: повторяются только те ошибки, которые имеет смысл повторять (429, 5xx, временные сетевые сбои), а на 4xx или отменённый контекст ресурсы не тратятся. Задержки ограничены, чтобы избежать лавинообразного роста повторных попыток. Ленивая инициализация откладывает создание каждого провайдера до первого реального использования — осознанное проектное решение, благодаря которому регистрация всех 43 провайдеров практически не требует затрат.

Модуль включает 43 готовых пакета провайдеров, а также универсальный адаптер, совместимый с OpenAI, который реализует полный интерфейс для *любой* конечной точки /v1/chat/completions — с поддержкой Bearer-аутентификации и потоковой передачи SSE с корректной обработкой сигнала [DONE]. Это позволяет любому вендору без выделенного пакета стать полноценным участником системы сразу после указания адаптеру его URL. Учётные данные разрешаются в одном месте (apikeys с использованием строгого соглашения ApiKey\_<Provider>), что исключает целый класс ошибок типа *«жёстко закодированный ключ проходит тесты, а реальный ключ не подключён — и всё ломается в продакшене»*.

Обнаружение моделей реализовано предельно честно: модуль запрашивает актуальные данные у API провайдеров через кэш с TTL, а старая система жёстко закодированных запасных

вариантов была полностью удалена по соображениям управления. Если живой запрос не удаётся, LLMProvider не возвращает ничего вместо устаревшего каталога, чтобы вызывающая сторона никогда не получала идентификатор модели, который выглядит валидным, но не может быть использован. Вся функциональность реализована потокобезопасно для параллельного использования.

## Почему мы это создали

Наивные вызовы LLM в продакшене проваливаются — провайдеры ограничивают частоту запросов, снижают качество обслуживания или вовсе выходят из строя, а один неисправный бэкенд может потянуть за собой весь сервис. Каталоги моделей устаревают, а жёстко зашитые списки передают вызывающим сторонам идентификаторы, которые уже не работают. LLMProvider централизует интерфейс, шаблоны отказоустойчивости и честное обнаружение, чтобы каждый потребитель автоматически получал устойчивость к сбоям и достоверность данных.

## Почему это меняет правила игры

Это сводит интеграцию провайдера LLM к одному действию — реализовать один интерфейс или просто направить универсальный адаптер на конечную точку — а затем автоматически и прозрачно оборачивает этого провайдера в механизмы размыкания цепи, мониторинга работоспособности и повторных попыток с экспоненциальной задержкой и джиттером. Отказоустойчивость перестаёт быть тем, что каждая команда изобретает заново (плохо, в условиях дедлайна, после первого инцидента), и становится поведением по умолчанию для всех 43 бэкендов. Надёжность пишется один раз, тщательно тестируется и достаётся бесплатно всем, кто импортирует библиотеку.

## Что здесь инновационного

- **Единый интерфейс с учётом возможностей** — завершение, потоковая передача, проверка работоспособности, возможности и валидация конфигурации объединены в один контракт, которому одинаково следуют все бэкенды.
- **Прозрачная обёртка размыкателя цепи — включая потоки.** Размыкатель защищает канал CompleteStream, а не только запросы/ответы, и воспринимает пустой поток как реальный сбой — с безопасным уведомлением слушателей без блокировок и в обход мьютексов.

- **43 пакета для провайдеров + универсальный адаптер, совместимый с OpenAI** — специализированные пакеты остаются лёгкими, а любой не внесённый в список вендор, поддерживающий `/v1/chat/completions`, начинает работать сразу, как только вы направите на него адаптер.
- **Единый источник учётных данных (apikeys)** — только одно место считывает переменные окружения `ApiKey_<Provider>`, структурно устраняя несоответствие «зелёные тесты — сломанный продукт» вместо того, чтобы просто предупреждать о нём.
- **Честное обнаружение моделей (без жёстко зашитых резервных вариантов)** — работающие API провайдеров за кэшем с TTL; при сбое возвращается `nil`, а не устаревший или сфабрикованный каталог, который выдаёт нерабочие идентификаторы.
- **Ленивая инициализация с `sync.Once`** — создание откладывается до первого использования, поэтому регистрация всех 43 провайдеров почти ничего не стоит, пока вы не вызовете хотя бы одного.
- **Антиобманный стек Challenge с поддержкой нескольких локалей** — настоящий раннер, который проверяет работу размыкателя цепи, мониторинга работоспособности и механизма повторных попыток в пяти локалях, контролируемый парным мутационным тестированием (немодифицированный код должен завершаться с кодом 0; внедрённая мутация должна приводить к коду 99), так что успешное прохождение тестов гарантирует корректность поведения.

## Главные технические вызовы и как мы их преодолели

- **Каскадные сбои провайдеров.** Один ненадёжный бэкенд не должен тянуть за собой весь сервис. Решение: трёхпозиционный размыкатель цепи (закрыто → открыто → полуоткрыто), который прозрачно оборачивает любого провайдера *и его поток*, срабатывает при устойчивых сбоях, проверяет восстановление в полуоткрытом состоянии и централизованно управляется `CircuitBreakerManager`.
- **Временные ошибки и ограничения частоты запросов.** Решение: экспоненциальная задержка с учётом статуса и джиттером —  $\min(\text{InitialDelay} \cdot \text{Multiplier}^{(n-1)}, \text{MaxDelay}) \pm \text{jitter}$  — чтобы повторные попытки распределялись во времени, а не синхронизировались в «стадо». Повторяются только те ошибки, которые должны повторяться (429, 500, 502, 503, 504 и сетевые ошибки), а бесполезные попытки при отменённом контексте или других 4xx отклоняются.
- **Масштабирование при большом количестве зарегистрированных, но неиспользуемых**

**провайдеров.** При 43 зарегистрированных провайдерах, из которых в каждом конкретном сервисе активно лишь несколько, ранняя инициализация была бы пустой тратой ресурсов. Решение: ленивая инициализация под защитой `sync.Once`, так что платить за настройку приходится только тем провайдерам, которые вы действительно вызываете.

- **Выдача недействительных идентификаторов моделей.** Решение: жёстко зашитый уровень резервного обнаружения был полностью удалён (согласно CONST-036), а при сбое живого обнаружения возвращается `nil` — плюс защитное копирование при возврате, чтобы вызывающая сторона не могла изменить кэш или создать состояние гонки с другим читателем. Достоверность обеспечивается структурно, а не по соглашению.
- **Потоковая передача и корректность конкурентного выполнения.** Скрытая проблема — взаимная блокировка между блокировкой размыкателя и его слушателями. Решение: создание снимка слушателей и их уведомление без блокировок с тайм-аутом в 5 секунд, а также снятие блокировки перед уведомлением о сбросе — при этом все компоненты рассчитаны на конкурентное использование и проверены с помощью флага `-race`.

## Технологический стек

- **Go (1.25.3)** — выбран за первоклассную поддержку конкурентности, статические бинарники и мощную стандартную библиотеку; включает модуль, интерфейс, все примитивы отказоустойчивости и все 43 адаптера.
- **net/http (stdlib)** — намеренно независимый от сторонних зависимостей HTTP: обеспечивает работу клиентов для каждого провайдера, универсальный адаптер, совместимый с OpenAI, и вызовы динамического обнаружения, так что не требуется проверять или обновлять сторонние транспорты.
- **logrus** — структурированное логирование с уровнями, дающее операторам необходимую прозрачность: фиксирует переходы состояний `circuit breaker` и этапы обнаружения.
- **testify** — управляет тестовым набором и, что особенно важно, механизмом привязки мутаций к веткам, благодаря которому успешное прохождение тестов действительно что-то значит.
- **yaml.v3** — парсит пакеты интернационализации и конфигурации в формате, который остаётся удобным для ручного редактирования.
- **digital.vasic.models** — общие типы `LLMRequest` / `LLMResponse` / `ProviderCapabilities`, собранные в одном месте, чтобы все адаптеры использовали единую терминологию (документированная зависимость времени исполнения).
- **Собственные пакеты** — `circuit`, `health`, `retry`, `apikeys`, `discovery`, `providers/` (43 вендора + `generic`) и `il8n`:

поверхность отказоустойчивости и интеграции разбита на небольшие, независимо тестируемые модули вместо монолитной архитектуры.

- **.env + ~/api\_keys.sh (соглашение ApiKey\_<Provider>)** — единый и однозначный источник учётных данных, благодаря чему ключи подключаются одинаково и в тестах, и в продакшене.
- **Makefile race suite (-race -p 1) + Challenge runner** — основа для проверки на прочность: детектор гонок подтверждает корректность конкурентности, а Challenge runner тестирует реальное поведение системы в условиях хаоса, DDoS-атак, масштабирования, стресс-тестов, динамического обнаружения и сценариев без приостановки работы.

## Статус и замечания по прозрачности

- **Статус: бета-версия.** Развязанный переиспользуемый модуль; репозиторий GitHub открыт.
- **Лицензия: не определена.** Противоречия — в doc.go указана MIT, но присутствует файл LICENSE в стиле Apache-2.0 — уточните перед публикацией.
- LLMsVerifier — вышестоящий единый источник истины для канонического каталога моделей. Манифест helix-deps.yaml устарел (указано deps: [], хотя в документации заявлена зависимость от digital.vasic.models); «Уровень 2 (models.dev)» в разделе discovery — запланированный заглушечный вариант, пока не активен.

**Приоритетный уровень:** Helix-primary (инфраструктурный кластер LLM — развязанный переиспользуемый модуль). Уступает по приоритету HelixTrack.